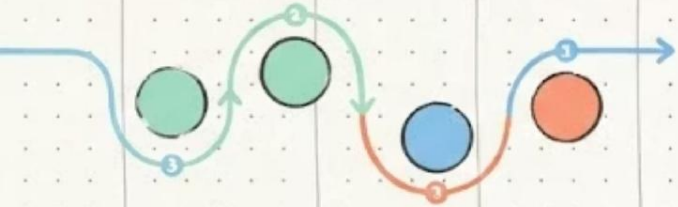


Gestão do Ciclo de Vida da Aplicação

DevOps

Professor Romário da Silva Borges



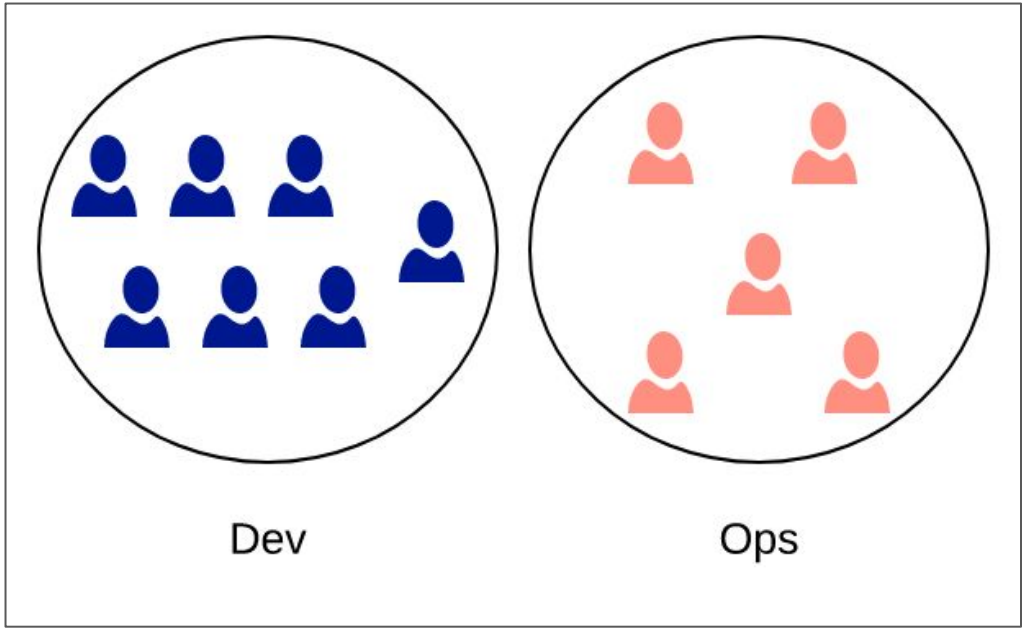
Nossa situação (no curso)

- Processo → organizar o desenvolvimento
- Requisitos → definidos e implementados
- Projeto e arquitetura → planejada e seguida
- Testes → implementados, com boa cobertura
- Refatorações → nunca foram deixadas de lado

Falta agora a "última milha":

implantar o sistema (colocá-lo em produção)

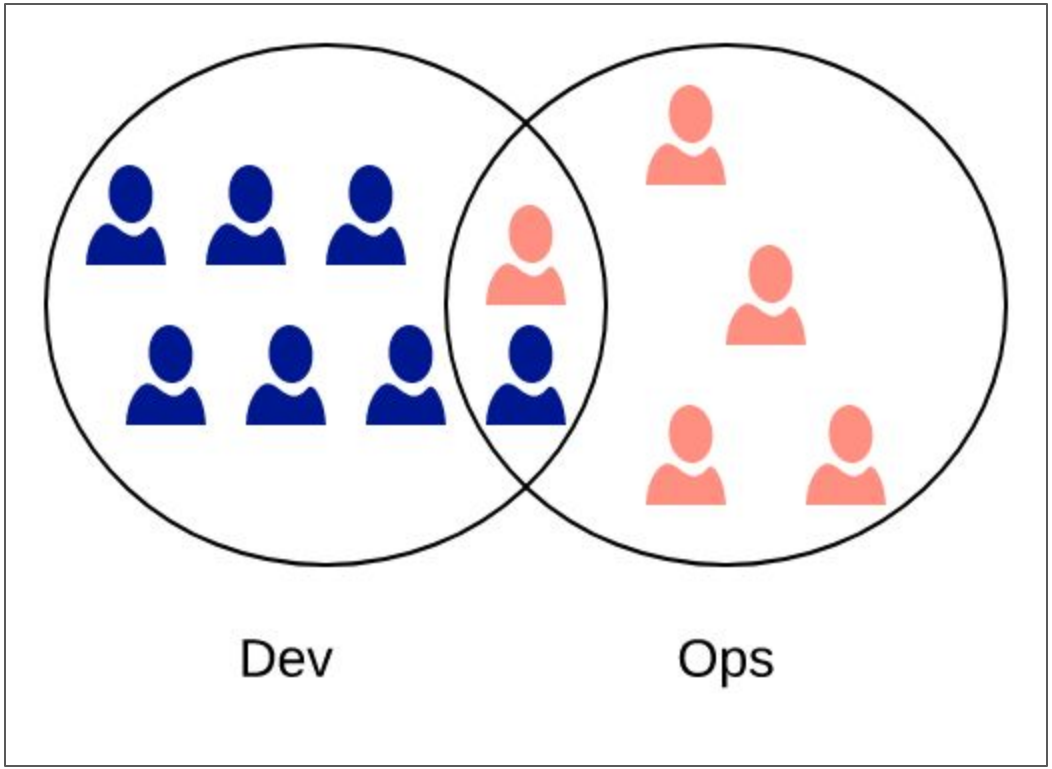
Antigamente, implantação era sempre traumática

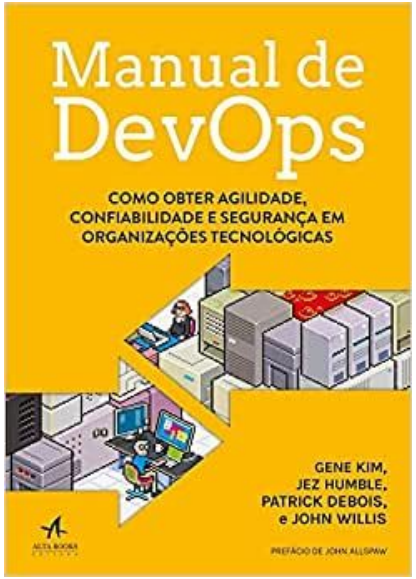


Dois silos independentes, com pouquíssima comunicação

Ops = administradores de sistema, suporte, sysadmin, pessoal de IT, etc

Ideia central de DevOps: aproximar Dev e Ops





Imagine um mundo no qual POs, devs, QA, "pessoal da TI" e de segurança trabalhem juntos ...

Objetivo: passagem de bastão bem sucedida!

(isto é, deve começar o quanto antes; ser automatizada, etc)

Ops



Dev

Objetivo: acabar com o jogo-de-empurra

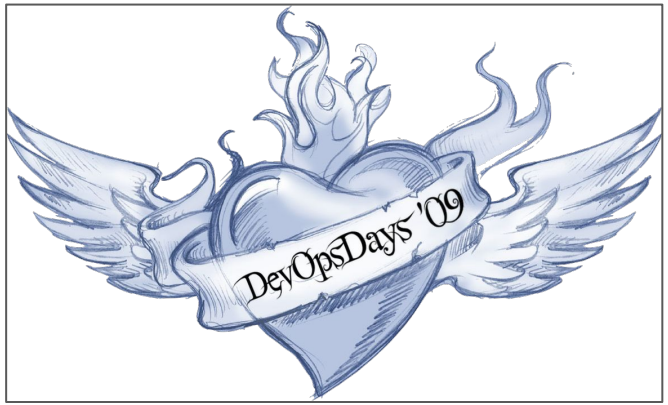
Dev: o problema não é meu código, mas seu servidor

Ops: o problema não é meu servidor, mas o seu código

"It works on my machine" ⇒ a comment that is incompatible with DevOps

DevOps

- Não é um cargo; mas um conjunto de princípios e práticas
- Termo surgiu ~2009



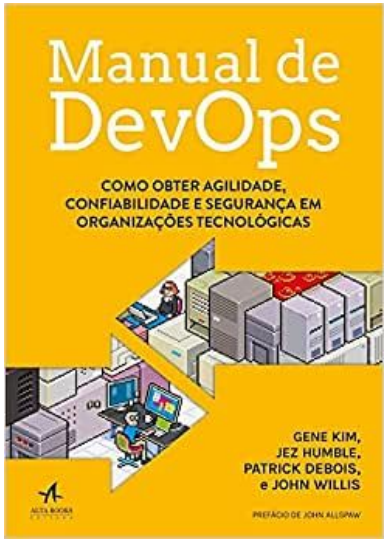
DevOps
(mentalidade ágil)

Incremental → implantar software todo dia

Automatização → VCS, testes, CI, CD

Princípios de DevOps

- Aproximar Devs e Ops, desde o início do projeto
- Adotar princípios ágeis na fase de implantação
- Transformar implantações em um não-evento
- Implantar sistemas (partes dele) todos os dias
- Automatização do processo de implantação



Em vez de iniciar as implantações à meia-noite de sexta-feira e passar o fim de semana trabalhando para concluí-las, as implantações ocorrem em qualquer dia útil, quando todos estão na empresa e sem que os clientes percebam ...

Práticas de DevOps

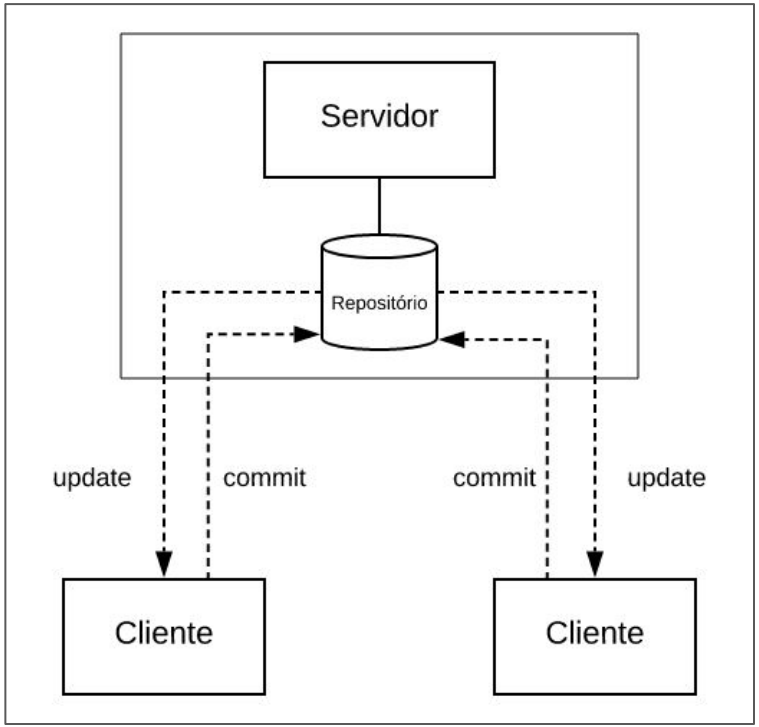
- Controle de Versões
- Integração Contínua
- Estratégias de Branch
- Deployment Contínuo
- Feature Flags

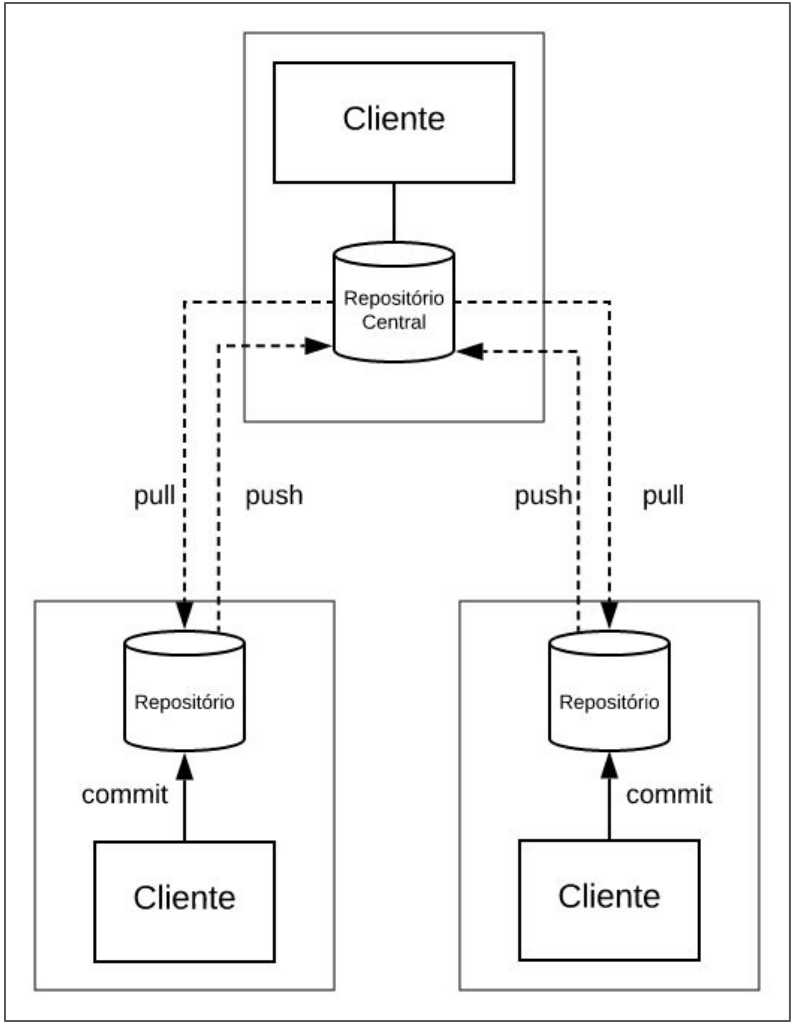
Controle de versões

Controle de Versões

- Fundamental para desenvolvimento colaborativo
- Armazena última versão do sistema (fonte da verdade)
- Permite recuperar versões anteriores

Centralizado (exemplo: svn, cvs)





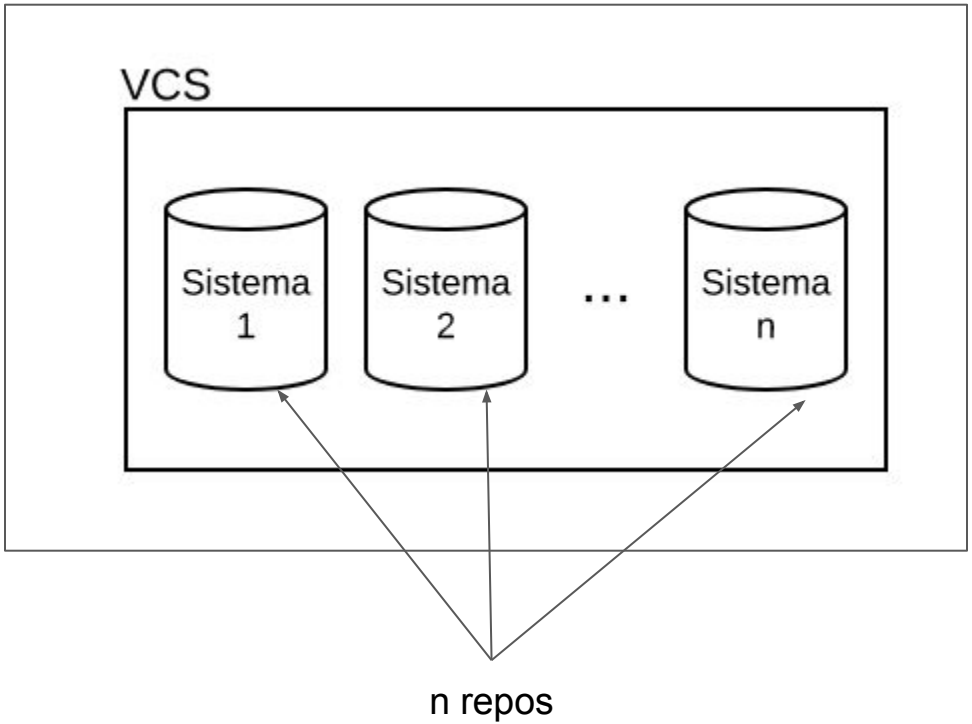
Distribuído
(exemplo: git, mercurial)

Vantagens de DVCS

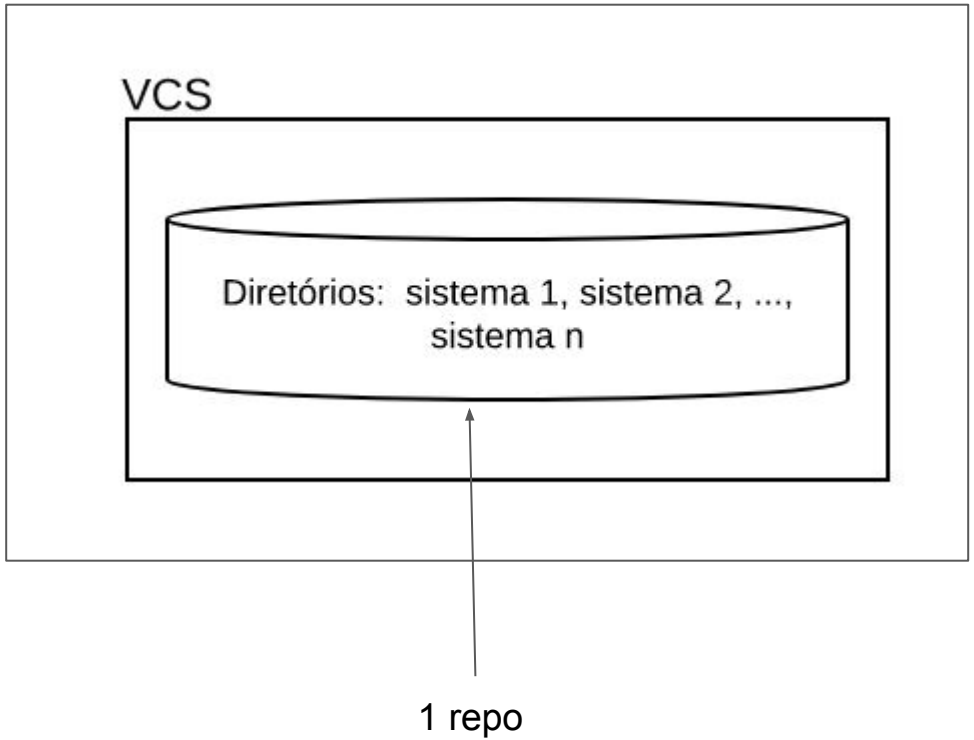
- Pode-se fazer commits com mais frequência
- Commits são mais rápidos
- Pode-se trabalhar offline
- Existe um histórico de versões local
- Arquiteturas alternativas: P2P, hierárquica, etc

Multirepos vs monorepo

Multirepo (mais comum)



Monorepo (um pouco menos comum)



Exemplo: GitHub

Multirepos

- unifal/sistema1
- unifal/sistema2
- unifal/sistema3

Monorepo

- unifal/sistemas
- Diretórios neste repo:
  - sistema1
  - sistema2
  - sistema3



contributed articles

DOI:10.1145/2854146

Google's monolithic repository provides a common source of truth for tens of thousands of developers around the world.

BY RACHEL POTVIN AND JOSH LEVENBERG

Why Google Stores Billions of Lines of Code in a Single Repository

This article outlines the scale of that codebase and details Google's custom-built monolithic source repository and the reasons the model was chosen. Google uses a homegrown version-control system to host one large codebase visible to, and used by, most of the software developers in the company. This centralized system is the foundation of many of Google's developer workflows. Here, we provide background on the systems and workflows that make feasible managing and working productively with such a large repository. We explain Google's "trunk-based development" strategy and the support systems that structure workflow and keep Google's codebase healthy, including software for static analysis, code cleanup, and streamlined code review.

Google-Scale

Google's monolithic software repository, which is used by 95% of its software developers worldwide, meets the definition of an ultra-large-scale[4] system, providing evidence the single-source repository model can be scaled successfully.

The Google codebase includes approximately one billion files and has a history of approximately 35 million commits spanning Google's entire 18-year existence. The repository contains 86TB[a] of data, including approximately

a Total size of uncompressed content, excluding release branches.

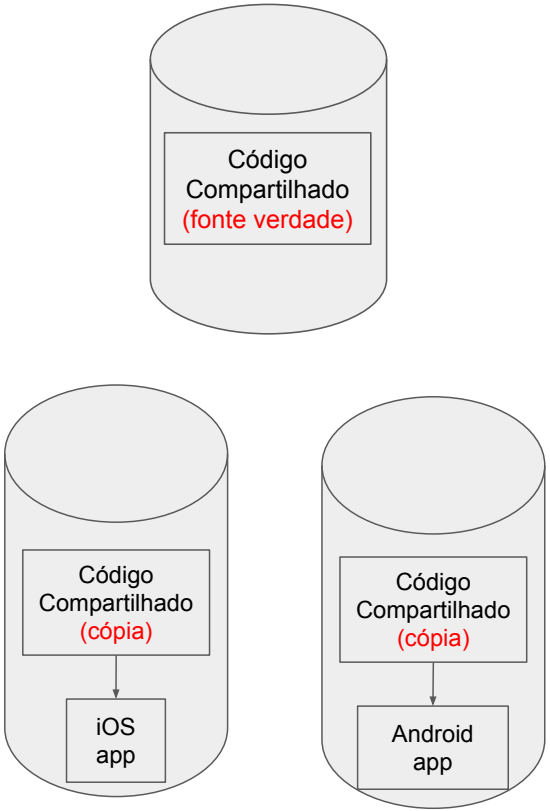
Monorepos são usados principalmente por grandes empresas

https://research.google/pubs/pub45424/

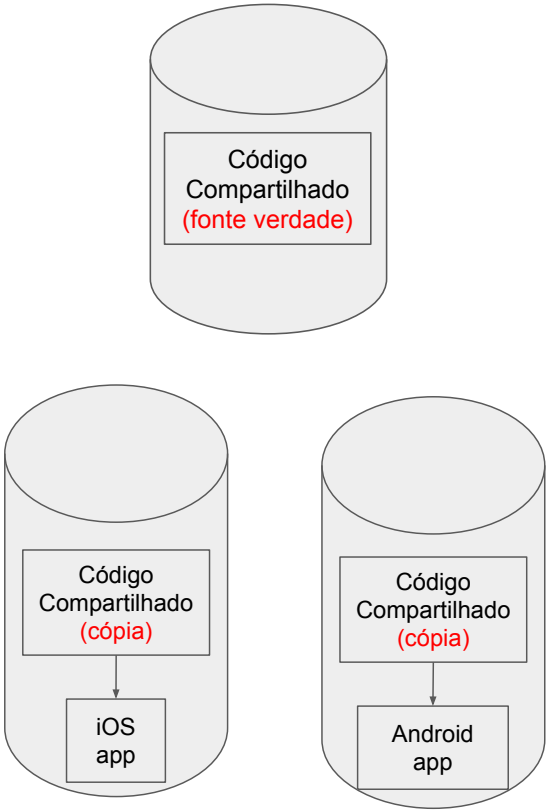
Vantagens de Monorepos

- Uma única fonte de verdade
- Promovem visibilidade e, portanto, reúso de código
- Mesma versão de uma biblioteca usada em todos subsistemas
- Mudanças são atômicas (1 commit pode alterar n sistemas)
- Facilitam refactorings em larga escala

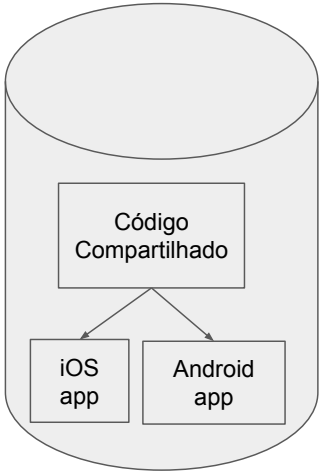
Multirepo
(3 repos)



Multirepo (3 repos)



Monorepo



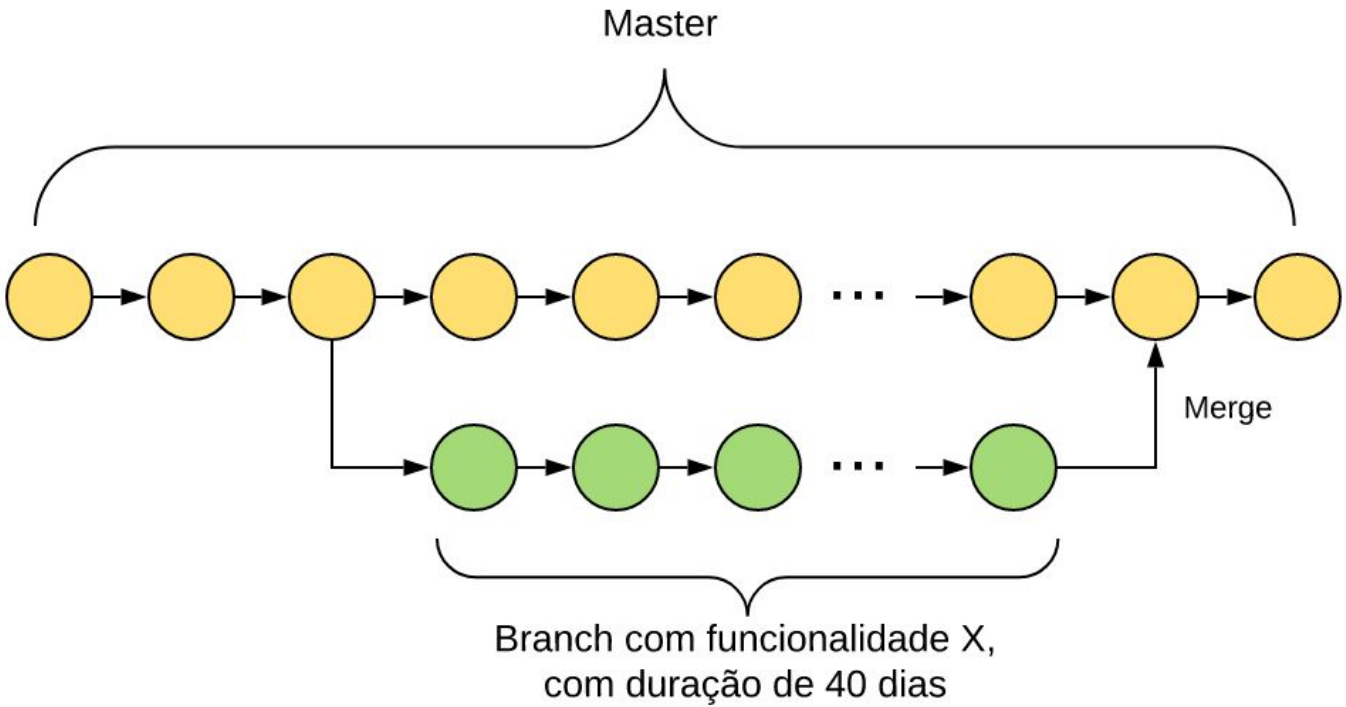
Desvantagem de Monorepos

- Podem requerer ferramentas customizadas (IDEs, etc)

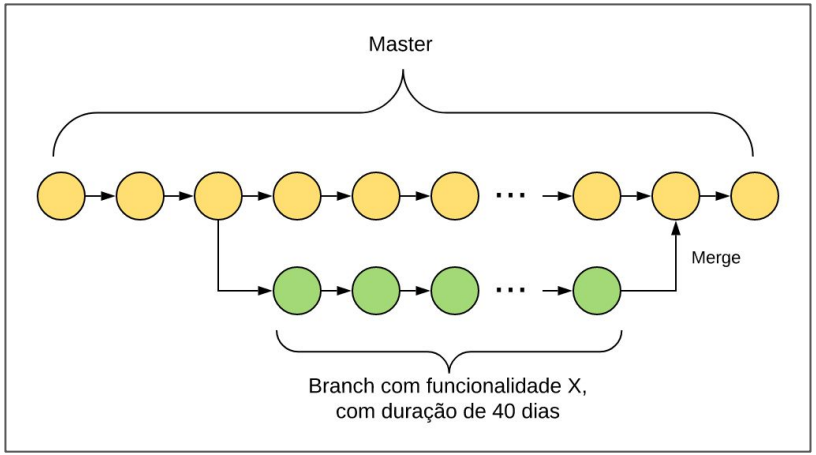
the size of the repository. For instance, Google has written a custom plug-in for the Eclipse integrated development environment (IDE) to make working with a massive codebase possible from the IDE. Google's code-indexing

Integração Contínua (CI)

Antigamente: feature branches eram comuns



Resultado após 40 dias: **merge hell**



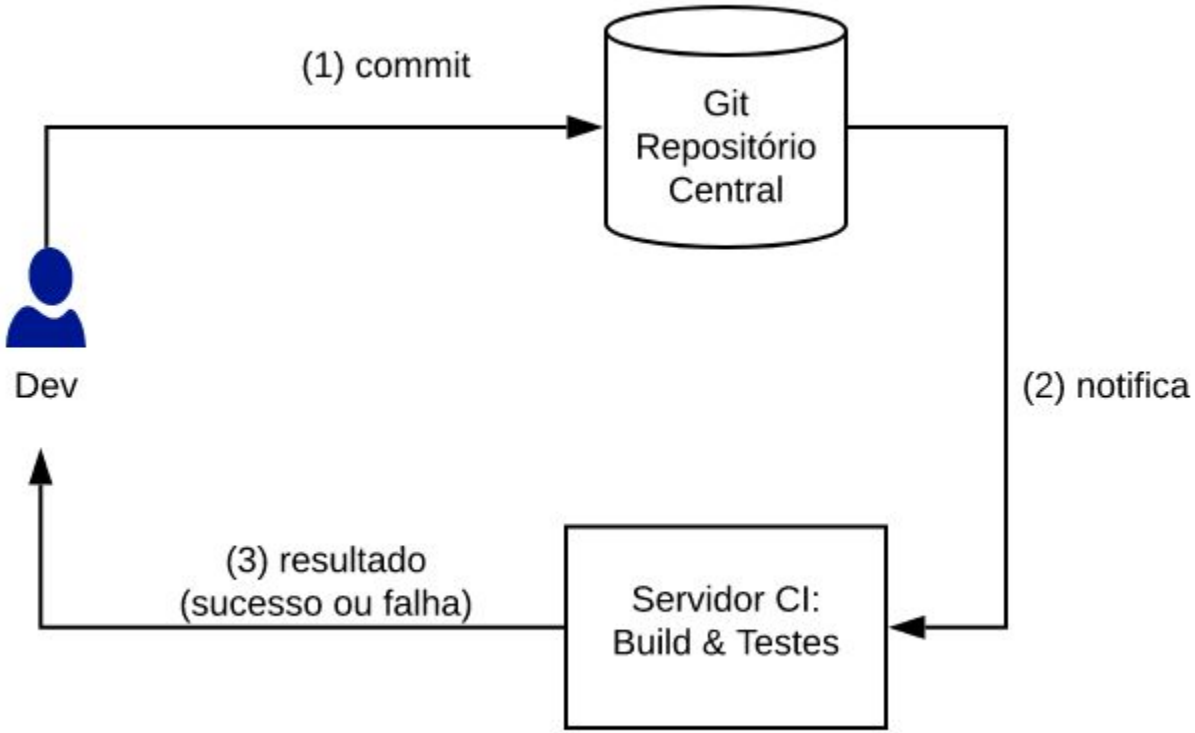
Integração Contínua (CI)

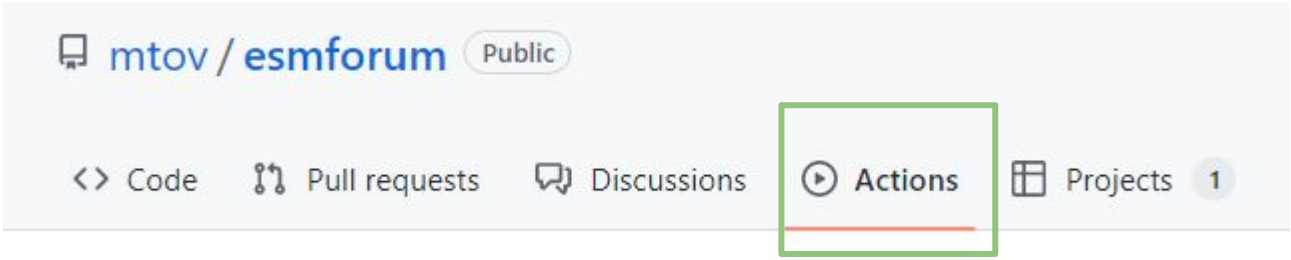
- Prática proposta por XP
- Como o nome diz, integrar o código com frequência
- Mas com qual frequência?
  - Não existe consenso, mas a maioria dos autores recomenda pelo menos uma vez por dia

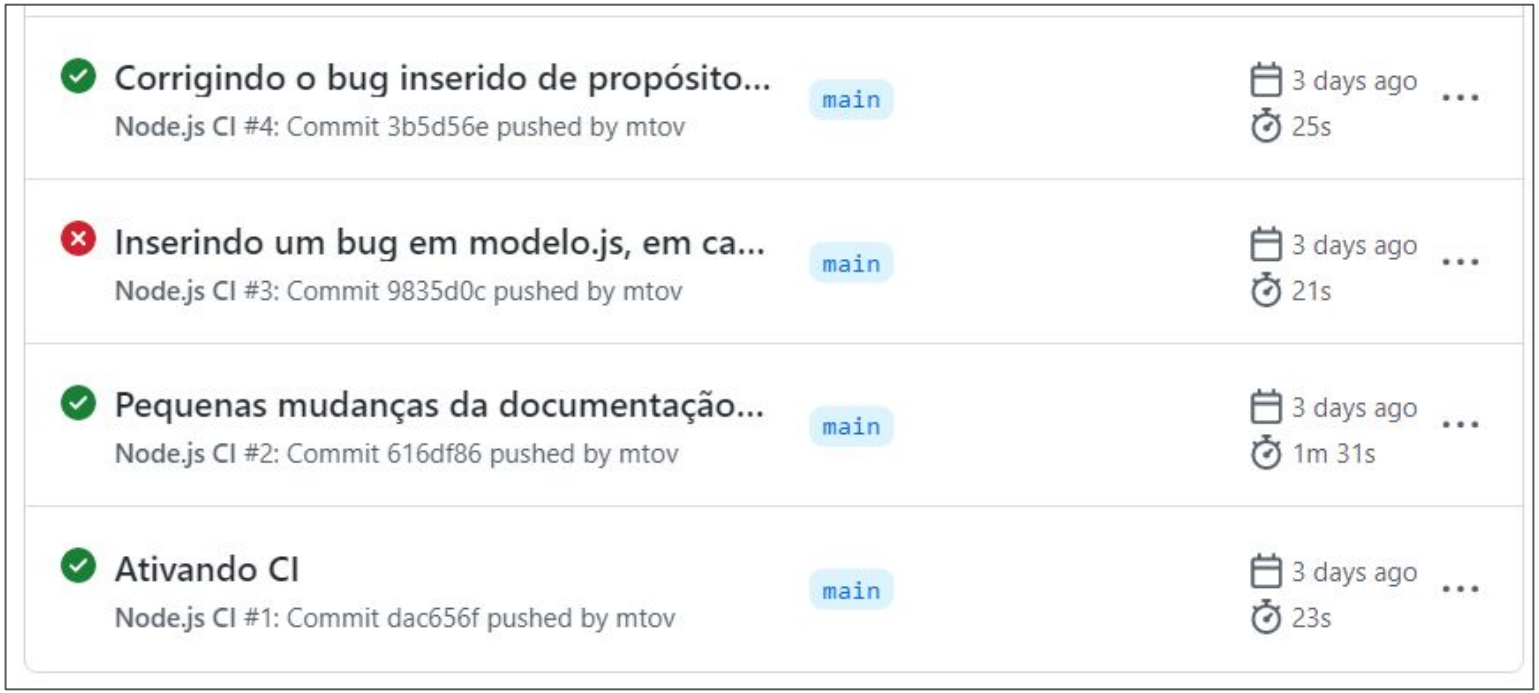
Boas práticas com CI

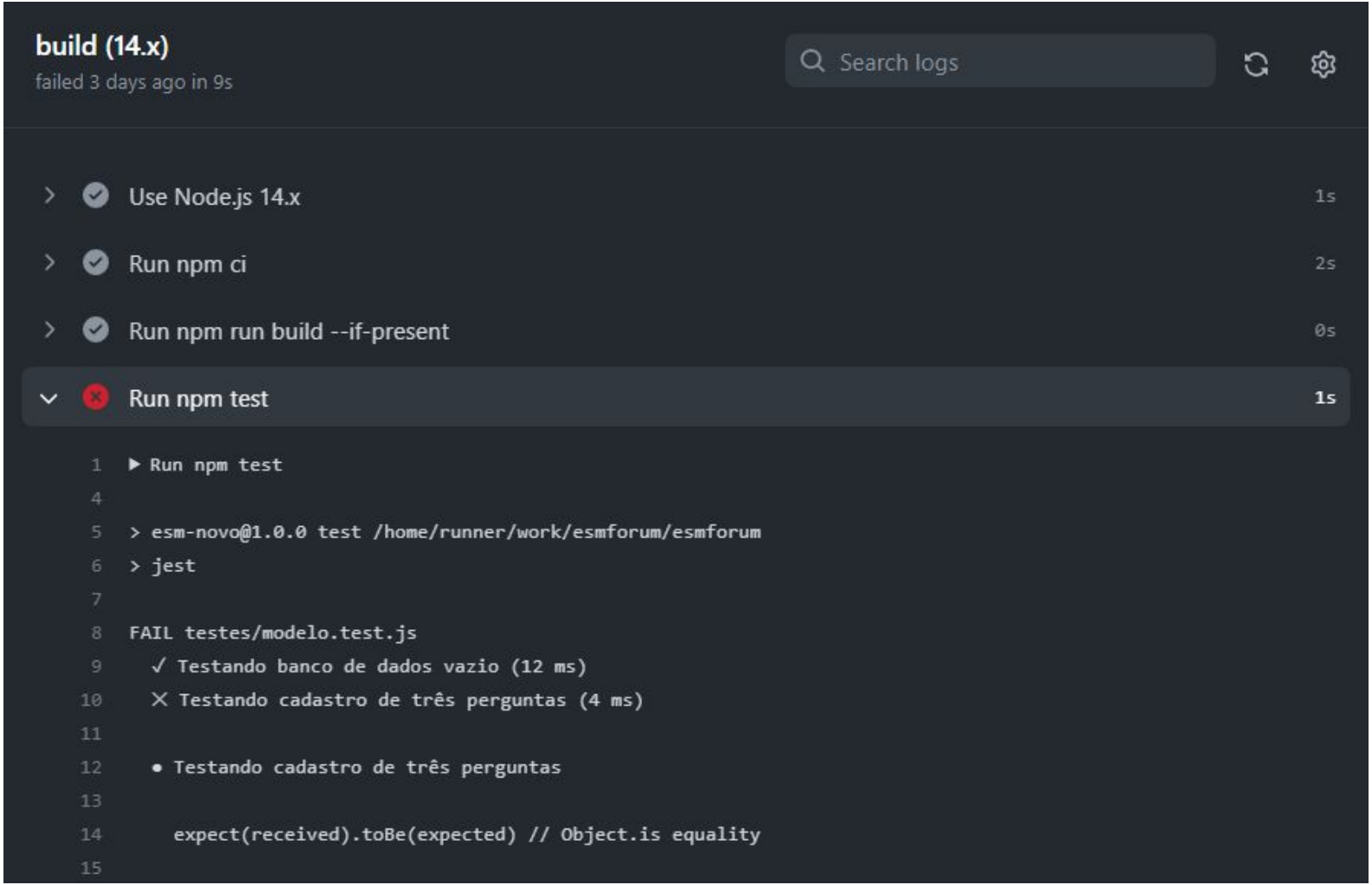
- Build automatizado
- Testes automatizados
- Programação em pares

Servidores de CI



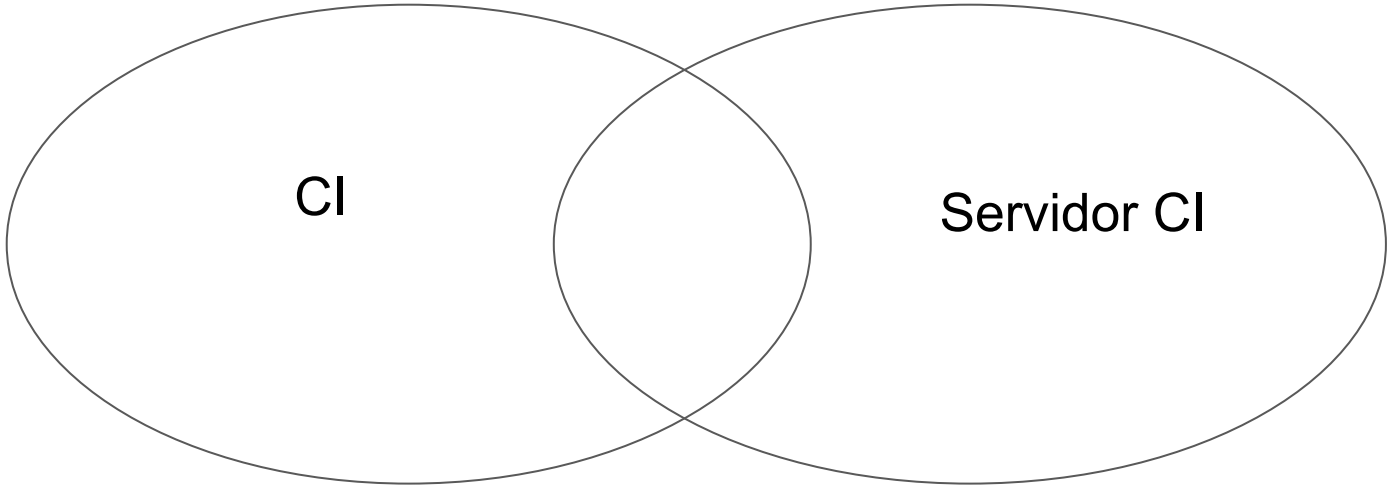

mtov / esmforum
Public
Code
Pull requests
Discussions
Actions
Projects
1


Corrigindo o bug inserido de propósito...
Node.js CI #4: Commit 3b5d56e pushed by mtov
main
3 days ago
25s
Inserindo um bug em modelo.js, em ca...
Node.js CI #3: Commit 9835d0c pushed by mtov
main
3 days ago
21s
Pequenas mudanças da documentação...
Node.js CI #2: Commit 616df86 pushed by mtov
main
3 days ago
1m 31s
Ativando CI
Node.js CI #1: Commit dac656f pushed by mtov
main
3 days ago
23s


build (14.x)
failed 3 days ago in 9s
Search logs
Use Node.js 14.x
1s
Run npm ci
2s
Run npm run build --if-present
0s
Run npm test
1s
1 ▶ Run npm test
4
5 > esm-novo@1.0.0 test /home/runner/work/esmforum/esmforum
6 > jest
7
8 FAIL testes/modelo.test.js
9 ✓ Testando banco de dados vazio (12 ms)
10 ✕ Testando cadastro de três perguntas (4 ms)
11
12 • Testando cadastro de três perguntas
13
14 expect(received).toBe(expected) // Object.is equality
15

Não confunda adotar CI com adotar apenas um servidor de CI

Empresas ou projetos que usam ….



Estratégias de branches

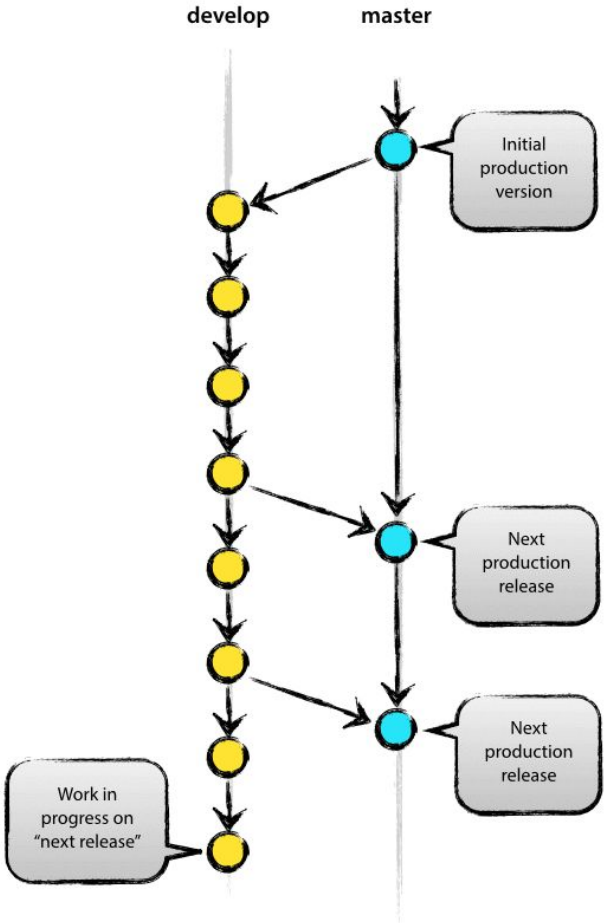
Estratégias de Branch

- Como organizar branches e gerenciar o fluxo de trabalho
- Principais estratégias:
  - Git-flow
  - GitHub Flow
  - Trunk-based Development

Git-flow

Git-flow

- Estratégia de branch muito comum
- Dois branches permanentes:
  - Main (ou master)
  - Develop



Fonte: https://nvie.com/posts/a-successful-git-branching-model/

Branches Permanentes

- Main (master ou trunk):
  - Código que está em produção
- Develop:
  - Funcionalidades que estão implementadas
  - Mas não passaram por um “teste final”
  - Por exemplo, pela equipe de QA

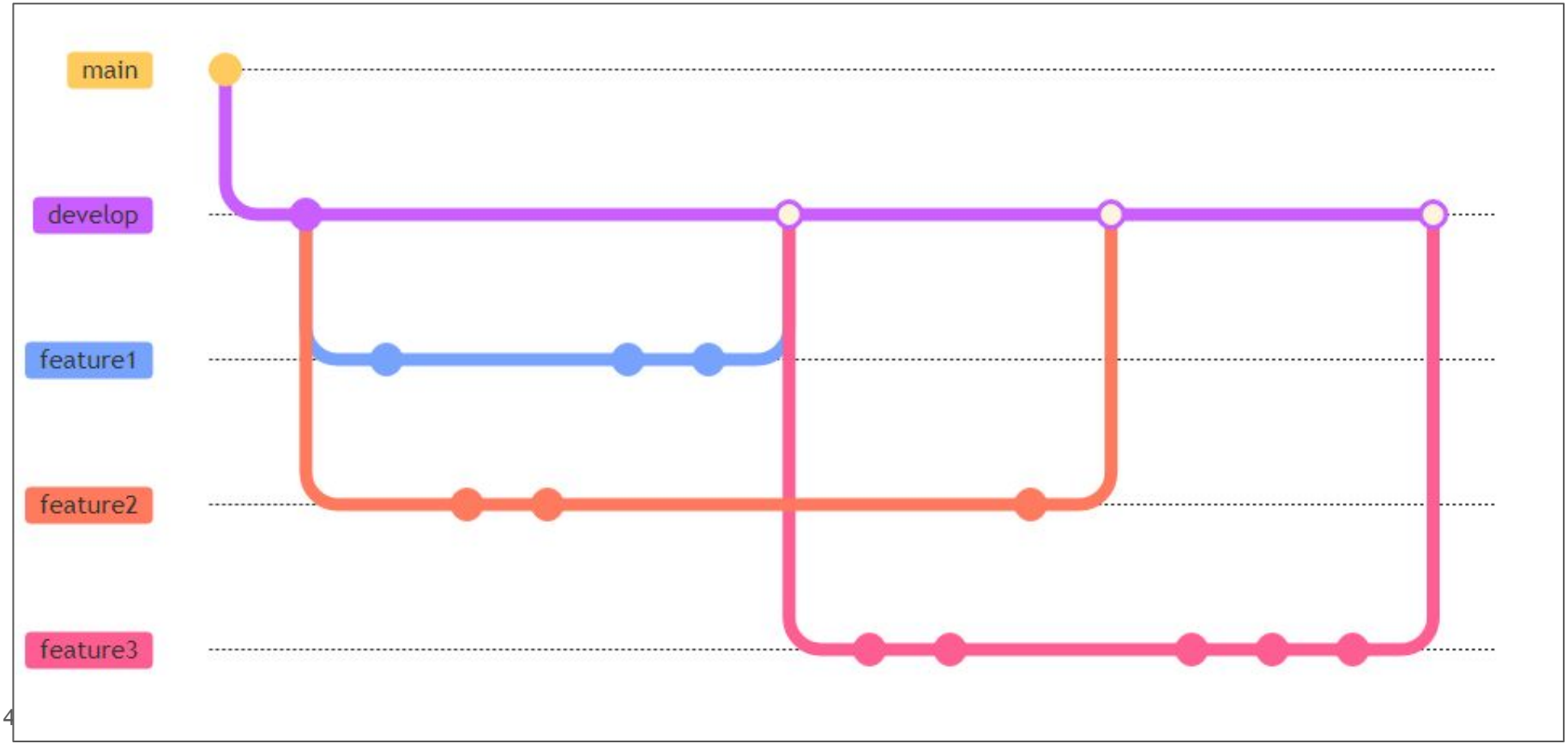
Branches de Apoio

- Três tipos:
  - Feature branches
  - Release branches
  - Hotfix branches
- Temporários (depois de um tempo, são apagados)

Feature Branches

- Branches para implementar uma nova feature
- Nascem de “develop”
- No final, merge de volta em “develop”
- Muitas vezes, existem apenas no repositório local de um desenvolvedor

Feature Branches



Comandos para criação de feature branches

```
git checkout -b nome-feature develop   % cria feature branch a partir de develop

[commits da feature]

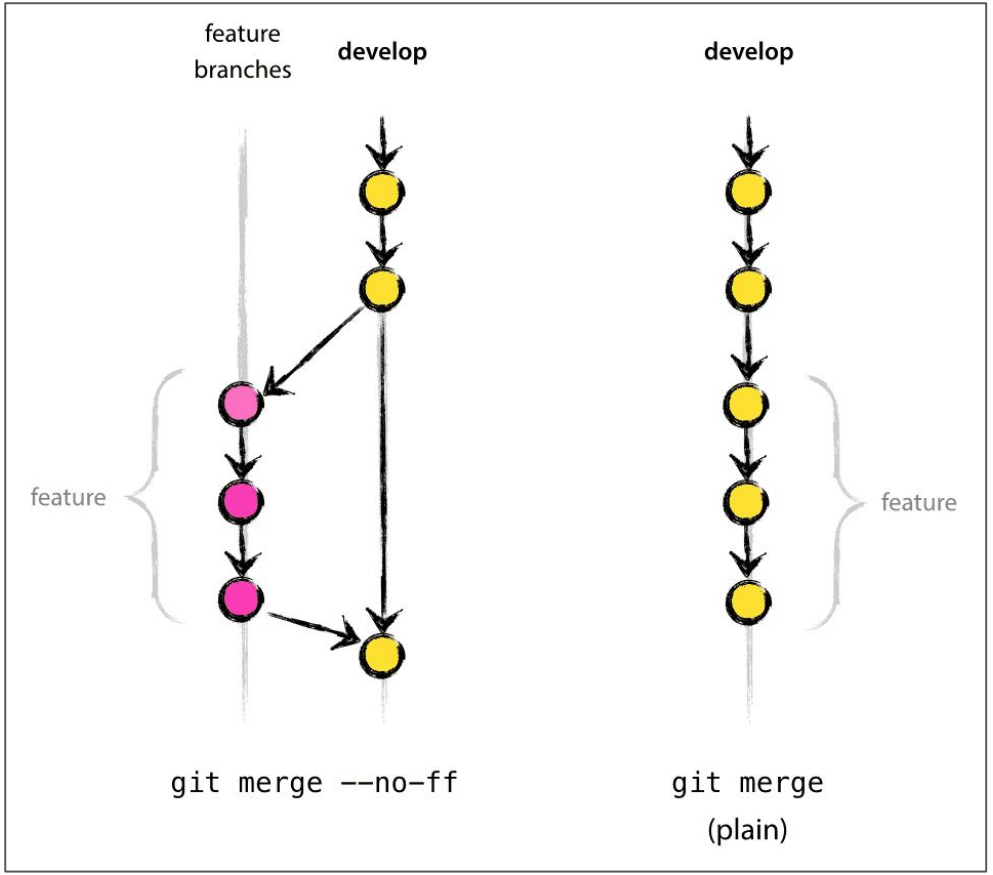
git checkout develop                   % volta para develop

git merge --no-ff nome-feature         % faz merge de nome-feature em develop
                                       % no-ff: no fast-forwarding (próx. slide)

git branch -d nome-feature             % remove feature branch

git push origin develop                % push de develop para repo remoto
```

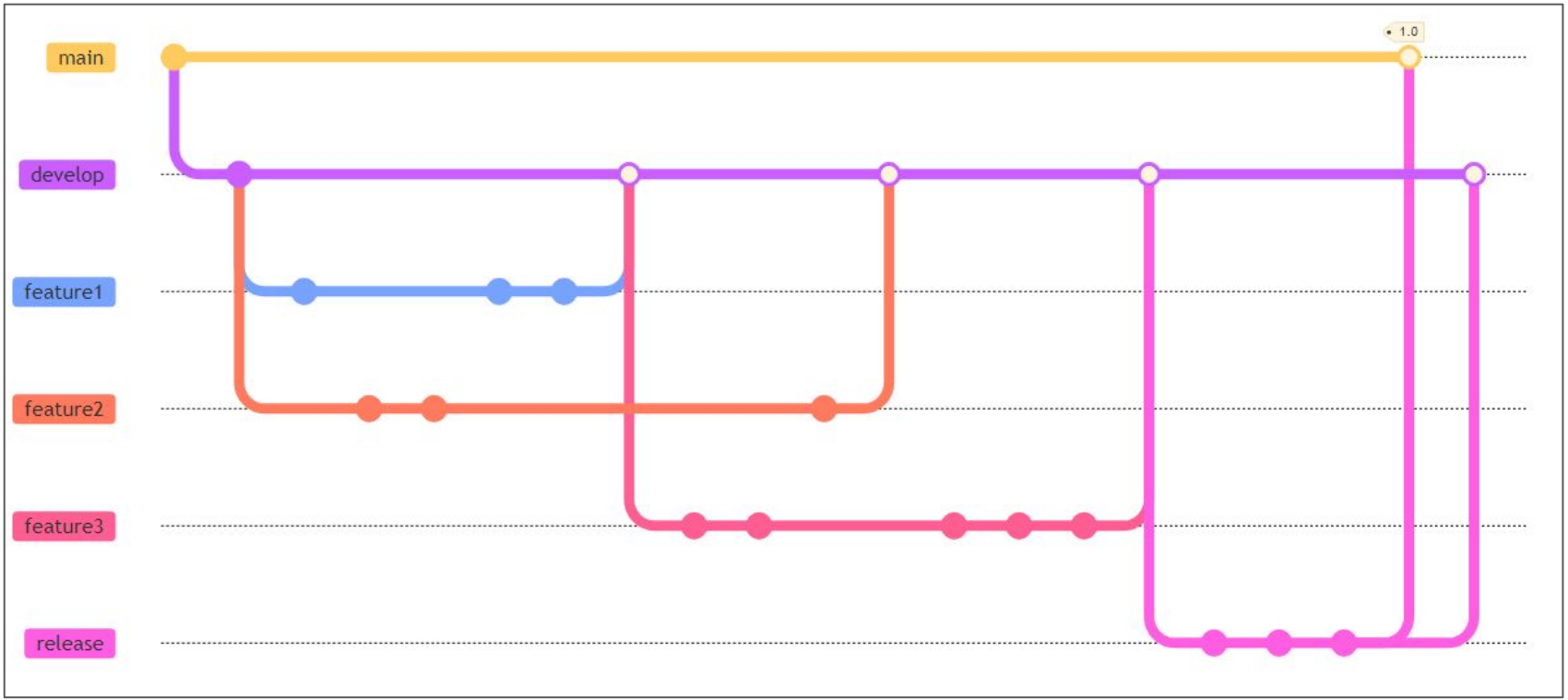
git merge: sem e com fast-forward



Release Branches

- Usados para preparar uma nova release, por exemplo, para ser aprovada por um cliente
- Nascem de develop
- No final:
  - merge no main (com a tag da nova release)
  - merge de volta em develop com bugs corrigidos

Release Branch (último branch da figura)



Comandos para criação de release branches

```
git checkout -b release-1.0 develop   % cria release branch a partir de develop

[commits da release]

git checkout master                   % move para master
git merge --no-ff release-1.0         % merge do release branch no master
git tag -a 1.0                        % adiciona tag no master

git checkout develop                  % move para develop
git merge --no-ff release-1.0         % merge do release branch em develop

git branch -d release-1.0             % remove release branch

git push origin develop               % push do branch develop
git push origin master                % push do branch master
```

Hotfix Branches

- Branches para corrigir bugs críticos detectado em produção
- Nascem do main (via tag da release onde bug foi reportado)
- Após correção:
  - merge no main (com nova tag)
  - merge também em develop

Hotfix Branches (último branch da figura)



Comandos para criação de hotfix branches

```
git checkout -b hotfix-1.2.1 master % cria branch do hotfix a partir do master

[commits do hotfix]

git checkout master                 % muda para o branch master
git merge --no-ff hotfix-1.2.1      % merge to hotfix branch no master
git tag -a 1.2.1                    % adiciona tag no master

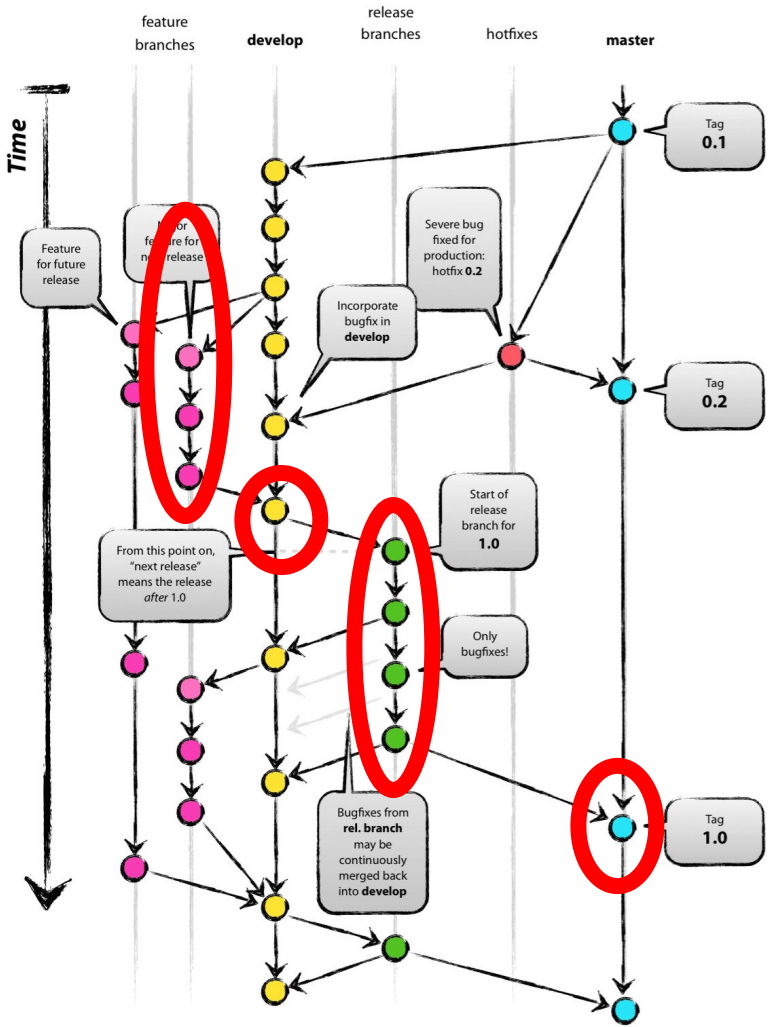
git checkout develop                % muda para o branch develop
git merge --no-ff hotfix-1.2.1      % merge do hotfix branch também em develop

git branch -d hotfix-1.2.1          % delta branch do hotfix

git push origin develop             % push de develop para repo remoto (github)
git push origin master              % push do master para repo remoto (github)
```

Git-flow: Resumo

Feature ⇒ Develop ⇒ Release ⇒
⇒ Main



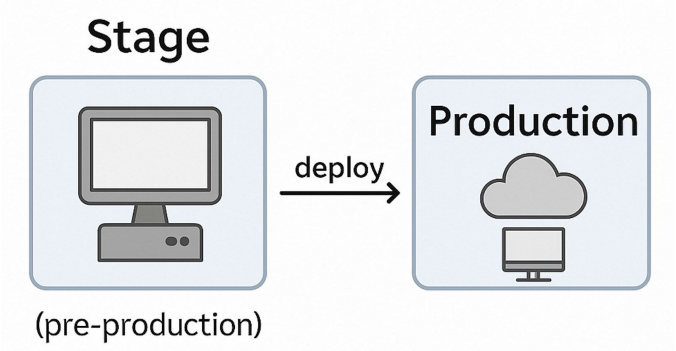
Git-flow: Uso e desvantagens

- Principal uso quando:
  - Vários clientes com versões diferentes
  - Testes manuais e times de QA
  - Releases precisam de aprovação dos clientes
- Desvantagens:
  - Tendência a ter merges maiores e com mais conflitos
  - E um ciclo de feedback dos clientes mais longo

Ambiente de Stage (homologação)

- Usado para testar o sistema antes de entrar em produção
- Ambiente → "máquina" (banco de dados, serviços externos, arquivos de configuração, usuários, permissões, etc)
- Deve ser o mais próximo possível do ambiente de produção

Outros nomes: ambiente de teste ou de homologação



Ambiente de Stage vs Release Branch

- Stage: um ambiente (uma máquina, cluster, instância, etc)
- Release Branch: um branch do Git, com o código que será testado no stage antes do deploy em produção

Exercícios

1. É possível usar Integração Contínua (CI) junto com Git-flow? Justifique.

Estratégias de Branch

https://engsoftmoderna.info/artigos/gitflow.html

Github Flow

GitHub Flow

- Fluxo comum quando se usa GitHub
- Git-flow simplificado:
  - Sem branches develop, release e hotfix
  - Apenas feature e main
- Mas com suporte a Pull Requests (PR)

Passos do GitHub Flow

- Desenvolvedor cria “feature branch” no seu repo local
- Implementa uma funcionalidade
- Faz um push do branch para o GitHub
- Entra no GitHub e abre um Pull Request (PR)
    - Pull Request: pedido para alguém revisar seu branch
- Revisor (outro dev) revisa e faz o merge do PR em “main”

Comandos git para uso de GitHub Flow

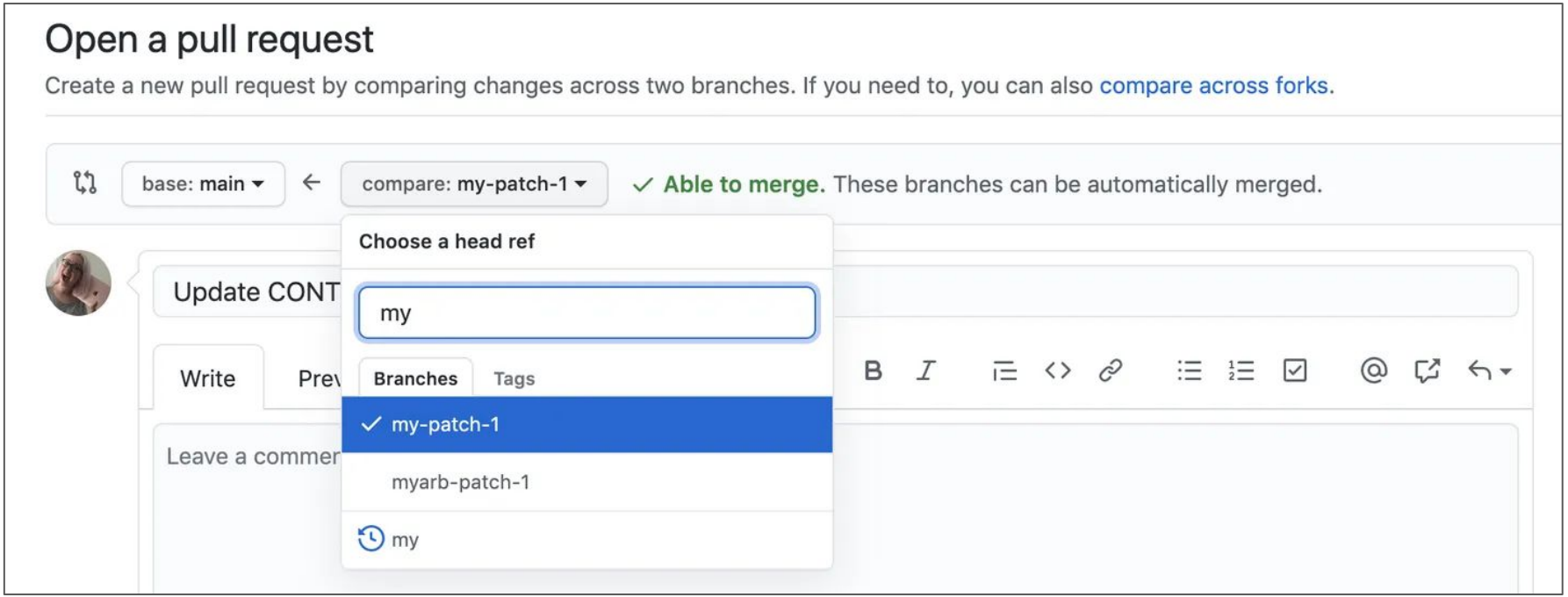
```
git checkout -b minha-feature    % cria feature branch locamente

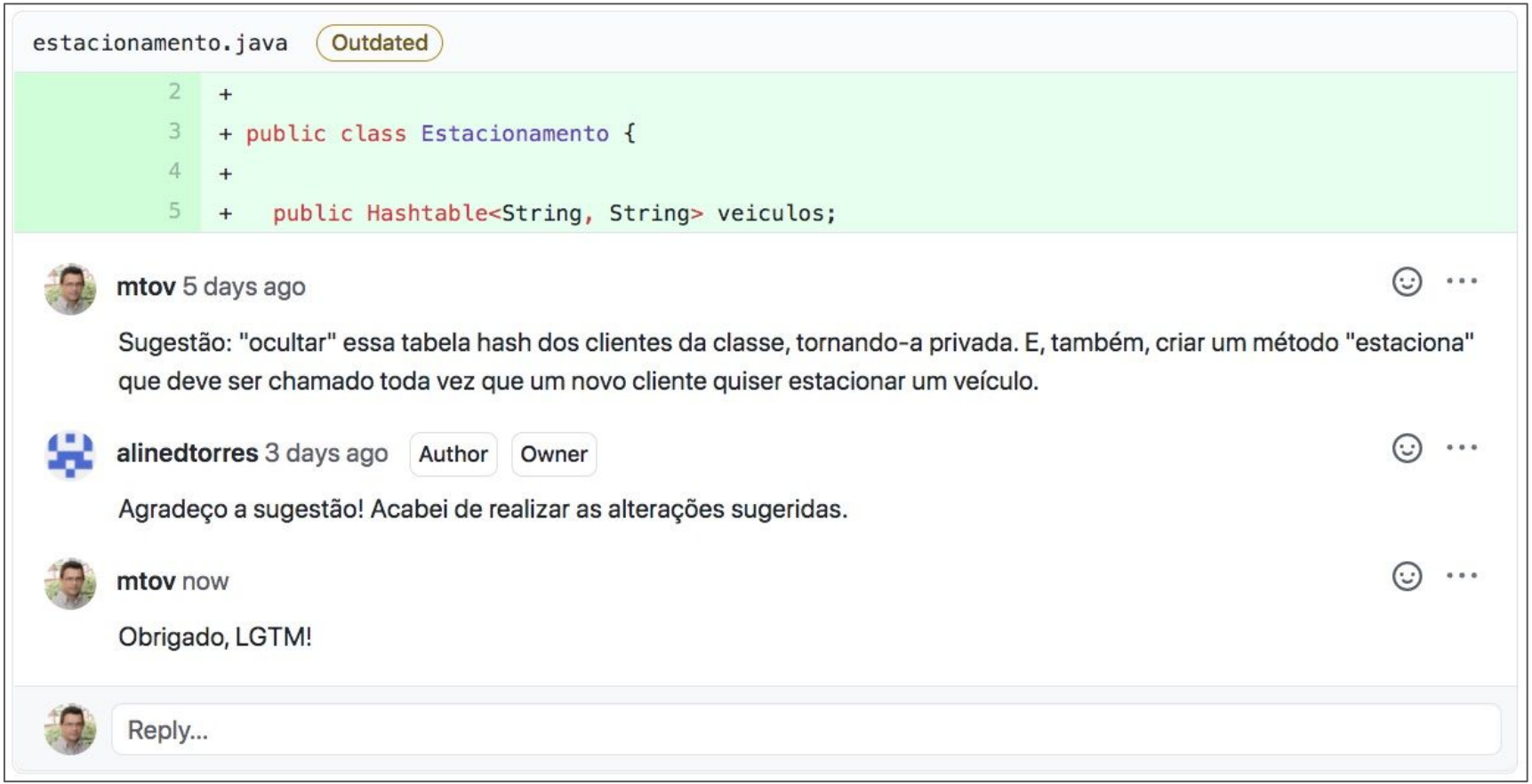
[commits da nova funcionalidade]

git add .
git commit -m "Minha nova funcionalidade"

git push origin minha-feature    % push do branch para o GitHub
```

Abrindo um Pull Request na Interface Web do GitHub




estacionamento.java
Outdated
2 +
3 + public class Estacionamento {
4 +
5 + public Hashtable<String, String> veiculos;
mtov 5 days ago
Sugestão: "ocultar" essa tabela hash dos clientes da classe, tornando-a privada. E, também, criar um método "estaciona" que deve ser chamado toda vez que um novo cliente quiser estacionar um veículo.
alinedtorres 3 days ago
Author
Owner
Agradeço a sugestão! Acabei de realizar as alterações sugeridas.
mtov now
Obrigado, LGTM!
Reply...

GitHub Flow: Uso e desvantagem

- Quando usar:
  - Sistemas com apenas uma versão em produção
  - Exemplo: sistemas Web
- Desvantagem:
  - PRs podem levar muito tempo para serem revisados

Desenvolvimento baseado no Trunk (TBD)

Desenvolvimento baseado no Trunk (TBD)

- Já que merges podem gerar conflitos, TBD defende:
  - Não usar mais branches de funcionalidades
  - Implementação ocorre diretamente no branch principal
- Branch principal = trunk, master, main



"Quase todo desenvolvimento ocorre no HEAD do repositório. Isso ajuda a identificar problemas de integração mais cedo e minimiza o esforço para realização de merges."



"Todos engenheiros trabalham em um único branch... o que torna o desenvolvimento mais rápido, pois não dispende-se esforço na integração de branches de longa duração no trunk."

Source: Software Engineering at Google (link), Development and Deployment at Facebook (link)

Grandes empresas de tecnologias

1. Testes automatizados
2. Microsserviços
3. Filas de Mensagens e Pub/Sub
4. Monorepos
5. TBD
6. CI/CD
7. Releases canários

Deployment Contínuo

Deployment Contínuo (CD)

- CI: merges realizados com frequência
- CD: código integrado entra imediatamente em produção
- Objetivo: experimentação e feedback!

Como evitar que minhas implementações parciais cheguem até os usuários finais?

Feature Flags

Enquanto a feature estiver em desenvolvimento!

```
featureX = false;
...
if (featureX)
   "aqui tem código incompleto de X"
...
if (featureX)
   "mais código incompleto de X"
```

Também chamadas de feature toggles

Quando código ficar pronto: habilita-se flag

```
featureX = true;
...
if (featureX)
   "aqui tem código incompleto de X"
...
if (featureX)
   "mais código incompleto de X"
```

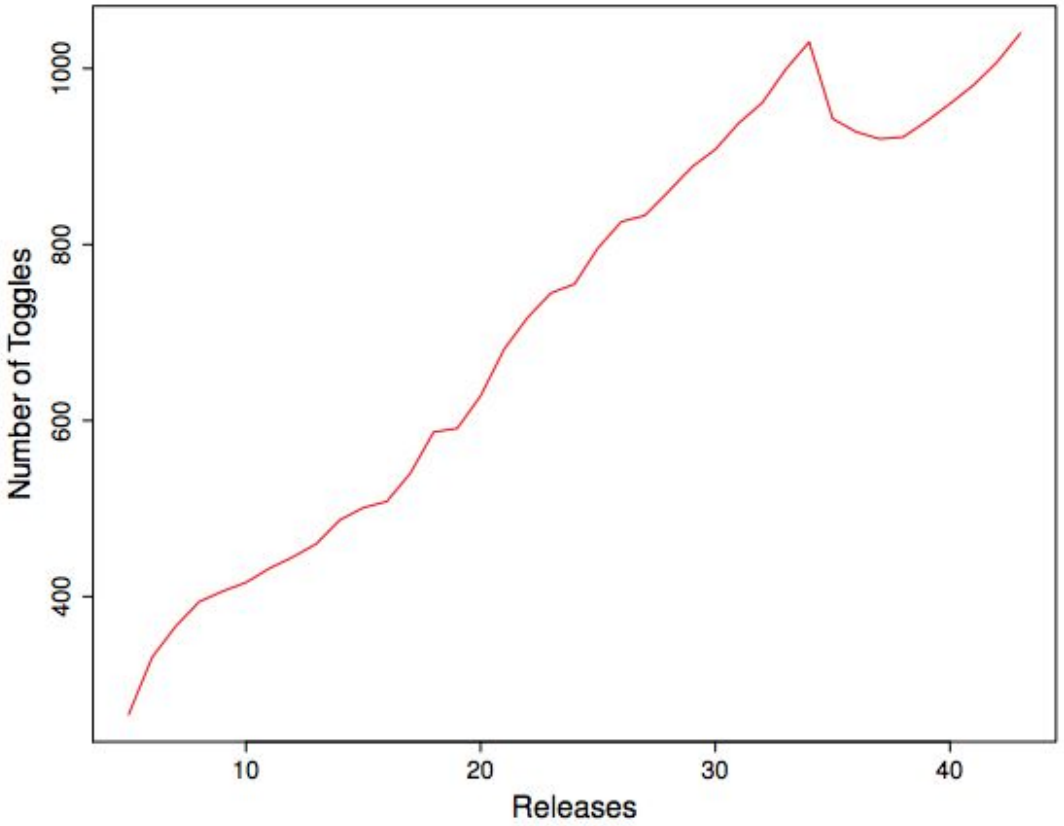


Figure 2: Number of unique toggles per release of Google Chrome.

Md Tajmilur Rahman et al. Feature toggles: practitioner practices and a case study. MSR 2016.

Rollout Gradual (Release Canário)

- Disponibilização ocorre de forma gradual, por exemplo:
  - Para um percentual pequeno de usuários
  - Ou para uma região ou país específico

Origem do termo release canário



Source: https://www.abc.net.au/news/2020-05-21/canary-coal-mine-gas-underground-mining-technology-monitoring/12259628

Exercícios

1. Suponha a seguinte função:

```
String destaca_texto(String texto, String palavra) {
    // “texto” é um texto em markdown
    // pesquisa todas as ocorrências de “palavra” em “texto”
    // converte palavra para negrito (**palavra**, em markdown)
}
```

Suponha que você está trabalhando em seu repositório local em um código que chama `destaca_texto`. Descreva uma mudança (push) feita nessa função, por outro dev, que:

(a) causa um erro de compilação no seu código (após um pull)?
(b) causa um erro de lógica no seu código (após um pull)?

2. Defina (e diferencie) os seguintes termos:

- Integração contínua (continuous integration)
- Entrega contínua (continuous delivery)
- Deployment contínuo (continuous deployment)



Integração
Contínua
(exemplo: diária)



Deployment Contínuo
(automático)



servidor de
produção



Integração
Contínua
(exemplo: diária)



Delivery Contínuo
(entrada em produção
deve ser manualmente
aprovada)



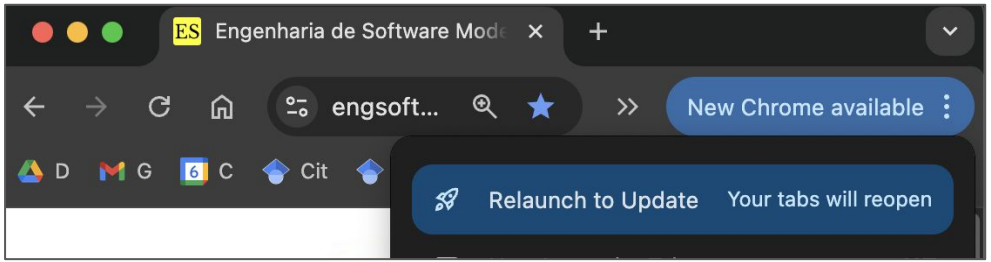
servidor de
produção

3. Suponha que você foi contratado por uma empresa que fabrica impressoras.

E que ficou responsável por definir as práticas de DevOps adotadas no desenvolvimento dos drivers dessas impressoras.

Qual das seguintes práticas você adotaria nesse desenvolvimento: deployment contínuo ou delivery contínuo? Justifique.

4. Em um browser como o Chrome, é melhor usar Delivery ou Deployment Contínuo?



5. Qual o "melhor" tipo de sistema para usar Deployment Contínuo? Justifique.

6. No contexto de TBD, feature flags são usadas para desabilitar implementações que ainda não estão prontas.

Porém, fora do contexto de TBD, feature flags podem ser usadas para habilitar ou desabilitar features “prontas” de um sistema. Dê um exemplo de um sistema e de algumas de suas features que podem ser ligadas ou desligadas.

8. Qual a diferença entre um teste A/B e um release canário?

Resumo: Feature flags/toggles podem ser usadas para:

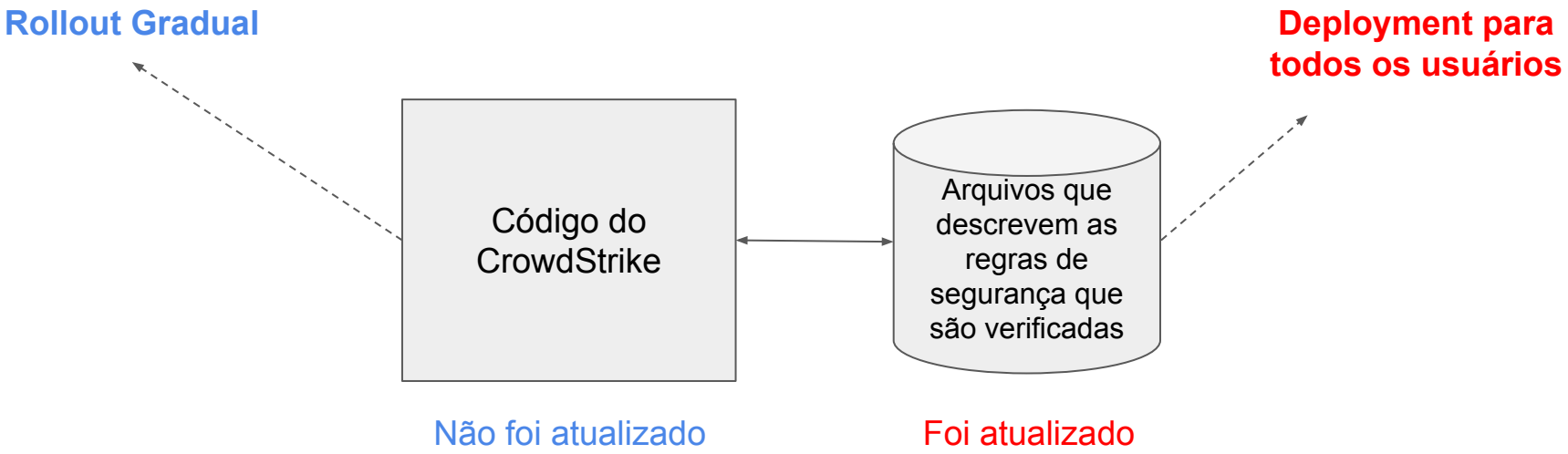
1. Continuous Deployment
2. Implementar releases canários
3. Ativar/desativar features opcionais (exemplo: sistemas com versões gratuitas e pagas)
4. Implementar testes A/B (objetivo: experimento que compara duas versões)

9. Complete supondo uma empresa que usa git-flow.

Tipo de Branch	Branch de Origem	Branch(es) de Destino
Feature		
Release		
Hotfix		

10. Em julho de 2024, uma falha em dos sistemas de proteção de ataques da empresa CrowdStrike resultou em "telas azul" em mais de oito milhões de máquinas Windows ao redor do mundo. Discorra sobre uma prática de DevOps poderia ser usada para evitar esse incidente?




Rollout Gradual
Deployment para todos os usuários
Código do CrowdStrike
Arquivos que descrevem as regras de segurança que são verificadas
Não foi atualizado
Foi atualizado



Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: https://engsoftmoderna.info.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

